

软工2复习

免责声明：文档内容由AI整理，请仔细辨别

总述

考试大纲 - 软件工程与计算II (基于Samperson的重点整理)

第一部分：软件维护、演化及文档 (第20-23章)

1. 软件维护

- **重要性理解**：为何软件维护至关重要（新需求、环境变化、缺陷修复）。
- **可维护软件开发方法**：
 - 考虑可变性：需求易变性分析、为变更设计。
 - 降低维护困难：技术文档、代码可读性、需求跟踪链、回归测试基线。

2. 演化式生命周期模型

- **各阶段理解**：初步开发、演化（特征：可演化性、业务价值）、服务、逐步淘汰、停止。
- **阶段转换条件**（特别是演化到服务或停止）。

3. 用户文档与系统文档

- **用户文档**：目的（参考指南/操作教程），组织模式（指导模式、参考模式），内容要点。
- **系统文档**：目的（系统维护），内容要点（配置、权限、故障、备份等）。

4. 逆向工程与再工程

- **定义与基本原理**：逆向工程（抽取需求设计，更高层抽象）。
- **定义与目的**：再工程（改进理解，提高可维护性、复用性、演化性）。
- **常见活动**：重文档化、重组结构、语言转换、数据结构修改。

5. 软件开发过程模型 (第22-23章)

- **软件生命周期模型**：基本阶段（需求工程 → 软件维护）。
- **不同软件过程模型的解释与比较【重点题型】**
 - **题型要求**：对给定场景，判断适用模型；掌握各模型特征、优缺点、适用场景。
 - **构建-修复模型**：特征、适用场景。
 - **瀑布模型（文档驱动）**：特征、优点、缺点、适用场景。
 - **增量迭代模型（需求驱动）**：特征、优点、缺点、适用场景。

- **演化模型（需求驱动）**：特点、优点、缺点、适用场景。
- **原型模型（需求驱动）**：特点（抛弃式原型）、优点、缺点、适用场景。
- **螺旋模型（风险驱动）**：特点（风险驱动、原型解决风险）、优点、缺点、适用场景。

6. 软件工程知识体系

- **知识域**：列出10个知识域（需求、设计、构造等）。

第二部分：软件工程基础与项目启动 (第1-5章)

1. 软件工程概述

- **定义**：系统化、规范化、可量化方法；对方法的研究。
- **发展特点 (1950s—2000s)**：【简答题型】各年代的核心特征。

2. 项目启动 (第三、四章)

- **团队管理**：
 - **管理方法**：团队章程、持续成功、和谐沟通、避免团队杀手。
 - **团队结构**：主程序员团队、民主团队、开放团队。
- **质量保障措施**：【结合实验说明】
 - 各阶段的保障活动（评审、度量、测试）。
- **配置管理活动**：【实验中如何进行】
 - **活动内容**：标识、版本管理、变更控制、审计、报告、发布。
 - **工具示例**（如SVN）。

3. 软件需求基础 (第五章)

- **需求定义**：三个层面（用户所需条件/能力；系统满足规定；文档化表述）。
- **需求的三个层次【重点题型】**
 - **题型要求**：实例分析，判断需求层次；给出例子。
 - **业务需求**：战略出发点，高层目标，为何开发。
 - **用户需求**：用户任务期望，系统能帮用户做什么，特性。
 - **系统级需求**：系统行为期望，与外界交互，开发人员需实现什么。
 - **理解用户需求到系统需求的转化过程**。
- **需求的类型【重点题型】**
 - **题型要求**：实例分析，判断需求类型；给出例子。
 - **总体分类**：项目需求、过程需求、系统需求（软件、硬件、其他）、不切实际期望。
 - **软件需求分类**：
 - **功能需求**：核心，用户价值，交互行为。

- **性能需求**：速度、容量、吞吐量、负载、实时性（能举例，理解PR1-PR7）。
- **质量属性**：可靠性、可用性、安全性、可维护性、可移植性、易用性（能举例QA1-QA6）。
- **对外接口**：硬件、软件、数据库接口，用户界面。
- **约束**：开发运行环境，法律法规，商业规则。
- **数据需求**：存储数据描述，实体关系，完整性，保持要求。

第三部分：需求分析与文档化 (第6-7章)

1. 用例图建立

- **参与者**：定义，特点。
- **步骤**：目标分析 → 寻找参与者 → 寻找用例 → 细化用例。
- **判断标准与常见错误**：用例粒度，业务价值，避免单个操作/步骤/片面作为用例。

2. 分析类图建立 (概念类图/领域模型)

- **核心**：关注问题域，非软件内部。
- **步骤**：识别候选类（名词分析）→ 确定概念类（状态+行为）→ 识别关联（名词+动词+名词）→ 识别重要属性。
- **关系**：主要用继承、组合、聚合、普通关联。

3. 系统顺序图建立 (交互图)

- **步骤**：确定上下文 → 找交互对象 → 添加消息。
- **符号**：同步/异步/返回消息，opt/loop/alt标签，监护条件。

4. 状态图建立

- **基本元素**：状态、状态转换、事件、活动。
- **步骤**：确定上下文 → 识别状态 → 建立状态转换 → 完善。
- **注意点**：开始/结束状态。

5. 需求文档化与验证 (第七章)

- **需求规格说明的必要性**：沟通，系统级需求列表。
- **用例与规格说明的区别**。
- **需求规格说明的错误判定与修正【重点题型】**
 - **要点**：技术文档写作（简洁、精确、易读、易修改）；需求书写（用户术语、可验证、可行）；文档模板、完备性、一致性、优先级。
 - 能够识别并修正不良需求描述。
- **功能测试用例设计【重点题型】**

- 基于给定的需求示例设计测试用例。

第四部分：软件设计、构造与测试 (第8-19章)

1. 软件设计概述 (第八章)

- 定义：过程与结果。
- 核心思想：复杂度控制（分解、抽象、层次性）。
- 设计层次：低层（数据结构+算法）、中层（模块化接口）、高层（体系结构）。

2. 软件体系结构 (第九、十章)

- 概念：部件、连接件、配置。
- 体系结构风格【重点】：
 - 主程序/子程序：部件、连接件、优缺点、实现。
 - 面向对象式：部件、连接件、优缺点、实现。
 - 分层风格：部件、连接件、优缺点、应用、实现。
 - MVC风格：模型、视图、控制器职责，连接件，优缺点、应用、实现。
- 体系结构设计过程：步骤1-7，关键决策（如安全与网络分布）。
- 包的原则：REP, CCP, CRP, ADP, SDP, SAP (理解各原则，CCP与CRP的权衡，ADP的解决)。
- 构件接口定义：如何从需求导出接口。
- 集成测试用例 (Stub和Driver)：定义，编写，持续集成过程。

3. 人机交互设计 (第十一章)

- 可用性：维度（易学、易记、效率、低出错、满意度）。
- 界面设计原则/注意事项【重点题型，至少5个】
 - 题型要求：判断例子违反的原则。
 - 例如：不暴露内部结构、所见即所得、常见界面类型。
- 精神模型与差异性：隐喻设计，用户分类（新手/专家）。
- 导航、反馈、协作式设计：全局/局部导航，响应时间，简洁/一致/低出错/易记性设计。

4. 详细设计基础 (第十二章) (中层+低层)

- 出发点：需求成果，体系结构成果。
- GRASP原则应用：
 - 信息专家：职责分配原则，优点。
 - 控制器：处理外部事件，解耦。
 - 创建者：对象创建职责分配。
 - 纯虚构：软件特有构造，作用。

- **职责分配与协作**：静态模型（属性/方法职责，关系），动态模型（职责聚合/分配）。
- **控制风格**：分散式、集中式、委托式。
- **设计类图/详细顺序图建立**：辅助类。
- **协作测试**：MockObject。

5. 模块化与信息隐藏 (第十三章)

- **耦合【重点题型】**：程度高低（内容、公共、重复(控制)、控制(印记)、印记(数据结构)、数据），能判断实例耦合类型。
- **内聚【重点题型】**：程度高低（信息、功能、通信、过程、时间、逻辑、偶然），能判断实例内聚类型。
- **信息隐藏基本思想【重点题型】**：隐藏设计决策（职责），契约与秘密，能判断实例信息隐藏好坏。
- 项目实践中的耦合、内聚、信息隐藏处理。

6. 面向对象详细设计原则 (第十四章)【重点题型】

- **题型要求**：对给定示例，发现违反的原则并修正。
- **原则列表**：无全局变量、显式、无代码重复、针对接口编程、迪米特法则 (LoD)、接口分离原则 (ISP)、Liskov替换原则 (LSP)、使用组合代替继承、单一职责原则 (SRP)。
- 理解各原则的目的（降低耦合、提高内聚）。

7. 面向对象信息隐藏 (第十五章)

- **信息隐藏含义**：隐藏需求与实现、实现机制变更。
- **封装**：数据与行为集中；接口与实现分离；封装内容（数据与行为、结构、其他对象、子类、潜在变更）。
- **开闭原则 (OCP)**：对扩展开放，对修改封闭，识别违反标志 (switch/if-else)。
- **依赖倒置原则 (DIP)**：高层不依赖底层，都依赖抽象；抽象不依赖细节，细节依赖抽象。

8. 软件构造与代码设计 (第17、18章)

- **构造活动**：详细设计、编程、测试、调试等。
- **名词解释**：重构、测试驱动开发 (TDD)、结对编程。
- **代码改进【重点题型】**：对给定代码，发现问题或改进（复杂决策封装、数据使用规范、明确依赖）。
- **契约式设计**：断言 (assert)，前置/后置条件。
- **防御式编程**：思想，与契约式设计的异同，编程方式。
- **表驱动**：适用场景。
- **单元测试用例设计**：MockObject，桩程序。

9. 可修改性、可扩展性、灵活性实现

- 方法：接口与实现分离，抽象类，委托机制。

10. 设计模式【重点题型】

- **题型要求**：给出场景应用模式写代码；给出代码用模式改写。
- **策略模式**：结构，职责，关系（组合），适用场景，相关原则，优缺点。
- **抽象工厂模式**：与简单工厂/工厂方法区别，产品族，适用场景，相关原则，优缺点。
- **单件模式**：实现方式，相关原则。
- **迭代器模式**：结构，行为，优点，相关原则，应用。
- 模式与信息隐藏、对象创建、接口编程的关系。

11. 软件测试 (第19章)

- **黑盒测试**：基于规格，方法（等价类划分、边界值分析、决策表、状态转换）。
- **白盒测试【必考】**：基于代码，方法：
 - 语句覆盖
 - 分支/条件覆盖
 - 路径覆盖 (能解释并区别这三种)。
- **测试用例设计场景【重点题型】**
 - 功能需求 → 功能测试用例。
 - 设计图 → 集成测试用例 (Stub, Driver)。
 - 方法描述 → 单元测试用例 (MockObject)。
 - JUnit基本使用。

通用复习建议：

- **理解概念**：不仅要记住定义，更要理解其内涵和应用场景。
- **区分比较**：对于相似或相关的概念（如不同过程模型、耦合与内聚、不同设计模式），重点比较其异同。
- **掌握题型**：特别注意标注了“【重点题型】”或“【必考】”的部分，针对性练习。
- **结合实例**：很多知识点需要通过实例来理解和应用，复习时多看例子。
- **实践联系**：思考这些理论知识在实际项目（如实验）中是如何体现的。

预祝考试顺利！

论述题

第1-5章: 绪论与项目启动

第1-2章: 软件工程定义与历史

1. 软件工程定义

- (1) 应用系统的、规范的、可量化的方法，来开发、运行和维护软件（即工程化应用到软件）。
- (2) 对(1)中各种方法的研究。

2. 1950s—2000s之间的特点 (简答)

- 1950s: 科学计算，机器为中心，软件生产像硬件。
- 1960s: 业务应用（批处理、事务），软件区别于硬件，软件工艺。
- 1970s: 结构化方法，瀑布模型，强调规则纪律（奠定基础）。
- 1980s: 追求生产力，现代结构化/面向对象，重视过程。
- 1990s: 企业级大规模系统，快速开发、可变更性、用户价值，Web出现。
- 2000s: 大规模Web应用/产品，快速开发、可变更性、用户价值、创新。
- 论述题考点：结合软件发展史，论述“软件危机”的产生原因及其对软件工程学科发展的推动作用。（虽然原文没直接提软件危机，但70年代的结构化方法和瀑布模型正是应对危机的产物）。

第3-4章: 项目启动

3. 团队

- 管理：团队章程，持续成功，和谐沟通，避免团队杀手。
- 结构：主程序员团队，民主团队，开放团队。
- 论述题考点：比较不同团队结构（如主程序员团队、民主团队）的优缺点及其适用场景。

4. 质量保障 (QA) 措施 (结合实验说明)

- 贯穿各阶段：
 - 需求开发：需求评审、需求度量。
 - 体系结构：体系结构评审、集成测试 (CI)。
 - 详细设计：详细设计评审、设计度量、集成测试 (CI)。
 - 实现阶段：代码评审、代码度量、测试 (TDD, CI)。
 - 测试阶段：测试执行、测试度量。
- 方法：评审、测试、质量度量。
- 论述题考点：论述软件质量保障活动如何在软件开发全生命周期中实施，并说明评审、测试和度量在QA中的不同作用。（可结合自己参与的实验项目，举例说明在需求、设计、编码、测试阶段分别采取了哪些QA措施）。

5. 配置管理 (CM) 活动 (实验中如何进行)
6. 标识配置项 (CI): 识别需要管理和控制的软件工作产品 (代码、文档、测试用例等)。
7. 版本管理 (Version Control): 追踪和管理配置项的变更历史。
8. 变更控制 (Change Control): 规范化变更请求、评估、批准和实施流程。
9. 配置审计 (Configuration Audit): 验证配置项的完整性和正确性, 确保与基线一致。
10. 状态报告 (Status Reporting): 记录和报告配置项状态和变更活动。
11. 软件发布管理 (Release Management): 管理和控制软件版本的发布。
 - 实验中: 通常使用版本控制工具如Git/SVN进行版本管理和基线管理, 可能通过Issue Tracker进行变更请求管理。
 - 论述题考点: 阐述软件配置管理的主要活动及其对保证软件产品一致性和可追溯性的重要性。结合一个配置管理工具 (如Git) 说明其如何支持版本控制和变更管理。

第五章: 软件需求基础

12. 需求定义
 - (1) 用户为解决问题或达目标所需的条件/能力。
 - (2) 系统/部件为满足合同、标准等需具备的条件/能力。
 - (3) 对(1)或(2)的文档化表述。
13. 区分需求的三个层次: 业务需求, 用户需求, 系统级需求
 - 业务需求 (Business Requirements):
 - 来源: 现实, 系统建立的战略出发点。
 - 表现: 高层次目标 (Objective), 为何开发系统。为满足业务需求, 定义系统应具备的特性 (Feature)。
 - 示例: R1 (提高销售额20%)。
 - 用户需求 (User Requirements):
 - 来源: 现实, 实际工作用户对系统能完成的具体任务的期望。
 - 表现: 系统能帮用户做什么。常模糊、混杂。
 - 示例: R2 (帮助收银员完成销售处理), UR1.1 (客户经理用系统增删改会员信息)。
 - 系统级需求 (System-level Requirements / Software Requirements):
 - 来源: 软件, 用户对系统行为的期望, 反映一次外界与系统交互或系统实现细节。
 - 表现: 开发人员需实现什么。是用户需求的细化和精确化。
 - 示例: R3 (收银员输商品, 系统显示描述、单价等), SR1.3.1 (接到请求, 系统提供所有会员个人信息)。

- 转化过程：用户需求 -> (分析问题域，建知识模型) -> 系统级需求 (定义系统行为联合实现用户需求)。
- 论述题考点：详细阐述业务需求、用户需求和系统级需求之间的区别与联系，并说明从用户需求到系统级需求的转化过程为何是需求工程中的关键活动。结合一个简单场景（如图书借阅）给出三个层次需求的实例。

14. 掌握需求的类型

- 大类：
 - 项目需求：关于项目本身的约束（如成本R5，时间R6）。
 - 过程需求：关于开发过程的要求（如提交文档R7，使用CI方法R8）。
 - 系统需求：针对系统本身（又分软件需求、硬件需求、其他需求R9, R10）。
 - 不切实际的期望：技术不可行R11，资源不足R12，超出软件影响范围R13。
- 软件需求分类：
 - 功能需求 (Functional Requirements)：系统应执行的活动，帮助用户完成任务，表现为系统与环境的交互。核心价值所在。
 - 性能需求 (Performance Requirements)：系统整体或部分的性能特征。
 - 类型：速度 (PR1查询<10s)，容量 (PR2存100万销售信息)，吞吐量 (PR3解析5000条/分)，负载 (PR4允许50并发)，实时性 (PR5报警<0.5s)。
 - 表达方式：可带概率 (PR6: 98%查询<10s)，或分级标准 (PR7最低/一般/理想)。
 - 质量属性 (Quality Attributes)：系统完成工作的“好坏程度”。
 - 类型：可靠性 (QA1网络故障系统不故障)，可用性 (QA2达到98%)，安全性 (QA3权限控制)，可维护性 (QA4增新特价类型<2月，含可修改、可扩展)，可移植性 (QA5换OS<1人月)，易用性 (QA6效率10件商品/分)。
 - 对外接口 (External Interfaces)：系统与其他系统（硬件、软件、数据库）的接口。
 - 约束 (Constraints)：构造系统时需遵守的限制（编程语言C1，硬件设施，法律法规，行业协定，商业规则R1/R2）。
 - 数据需求 (Data Requirements)：需存储的数据描述（功能使用数据，频率，访问性，实体关系，完整性，保持期DR1/DR2）。若功能需求已明确则可省略。
- 论述题考点：
 - 阐述功能需求与非功能需求（特别是性能需求和质量属性）的区别，并说明为何在需求获取和规格说明中同等重要。
 - 选择几个重要的质量属性（如可维护性、易用性、安全性），解释其含义，并说明如何将这些抽象的质量属性转化为可验证的需求描述。

- 针对一个给定的需求描述（可能混杂了不同类型的需求），识别并将其分类到功能需求、性能需求、质量属性、约束等类别中。

第6-7章: 需求工程

1. 建立用例图 (Use Case Diagram)

- 核心概念：
 - 参与者 (Actor): 与系统交互的用户或其他系统角色。一个参与者可代表多个用户，一个用户可扮演多种角色。参与者不一定是人。
 - 用例 (Use Case): 描述为应对一个业务事件，由一个用户发起，并在一个连续时间段内完成，可以增加业务价值的任务。
- 建立步骤：
 - a. 目标分析与解决方向确定：理解业务背景、痛点 (BRs - Business Requirements) 和期望的解决方案特性 (SFs - Solution Features)。
 - b. 寻找参与者：根据SFs和业务描述，识别出与系统交互的各种角色。
 - c. 寻找用例：从参与者的目标出发，确定他们需要系统完成哪些有业务价值的任务。
 - d. 细化用例：确保用例粒度合适。
 - 判断标准：用例描述了为应对一个业务事件，由一个用户发起，并在一个连续时间段内完成，可以增加业务价值的任务。
 - 避免：
 - 无业务价值的技术实现（如登录、数据校验、连接数据库）。
 - 无独立业务价值的单个操作（CRUD的单个字母）。
 - 将用例细化为单个操作、步骤或片面方面。
- 论述题考点：
 - 用例粒度的重要性及判断标准：论述为何用例粒度过大或过小都会影响后续分析设计，并详细解释如何判断粒度是否合适（业务事件、单一用户发起、连续时间、业务价值）。
 - 用例与非用例的界限：结合实例（如登录、数据校验）论述为什么某些活动不适合作为用例，它们应该在需求中如何体现（如作为非功能需求或业务规则）。
 - 如何从业务需求 (BR) 和系统特性 (SF) 推导出参与者和用例：结合示例1，阐述分析过程，如何从“痛点”、“期望”中识别出用户类别及其目标，进而转化为用例。
- 2. 建立分析类图 (Conceptual Class Diagram / Domain Model)
 - 核心概念：
 - 关注现实世界问题域，而非软件内部构造。
 - 对象 (Object): 标识（自治、协作）、状态（数据）、行为（操作数据）。

- 类 (Class): 对象的集合。
- 关联 (Association): 对象间潜在的链接 (交互路径)。
- 建立步骤:
 - a. 识别候选类 (名词分析法): 从需求描述中提取名词。
 - b. 确定概念类:
 - 既有状态又有行为 -> 概念类。
 - 仅状态无行为 -> 其他概念类的属性。
 - 仅行为无状态 -> 重新审视需求, 或剔除并将行为赋予有状态的类。
 - 无状态无行为 -> 剔除。
 - c. 识别关联: 提取 “名词+动词+名词” 结构, 依据需求和现实状况。
 - d. 识别重要属性: 为概念类添加必要的属性。
- 关系: 主要使用继承、组合、聚合、普通关联 (组合慎用, 聚合可不用)。
- 论述题考点:
 - 分析类图与设计类图的区别与联系: 论述两者在关注点、抽象层次、包含元素上的不同, 以及分析类图如何作为设计类图的基础。
 - 概念类的识别标准和过程: 详细阐述如何通过 “状态” 和 “行为” 两个维度来筛选和确定概念类, 以及如何处理那些不完全满足条件的候选类。
 - “一切看需求” 原则在分析建模中的体现: 结合示例中的 “商品ID”、“数量单位”、“价格” 等, 论述需求如何决定一个事物是否应被建模为具有状态和行为的独立概念类, 还是仅仅作为属性。

3. 建立系统顺序图 (System Sequence Diagram - SSD)

- 核心概念: 展现特定用例场景下, 参与者与系统之间的交互顺序, 系统被视为一个黑盒。
- 建立步骤:
 - a. 确定上下文环境: 明确是为哪个用例的哪个场景绘制。
 - b. 根据用例描述找到交互对象: 主要是参与者和系统。
 - c. 按照用例描述中的流程顺序逐步添加消息:
 - 消息类型: 同步消息、异步消息 (箭头样式一致)、返回消息。
 - 控制结构: opt (可选), loop (循环, 带条件), alt (选择, 带监护条件和虚线分隔)。
- 论述题考点:
 - SSD在需求分析中的作用: 论述SSD如何帮助精确描述系统行为、验证用例的完整性和一致性、以及作为后续设计的基础。

- SSD与用例描述的关系：阐述SSD是如何用例文字描述的图形化表示，以及如何通过SSD反过来完善用例描述的细节。

4. 建立状态图 (State Diagram)

- 核心概念：描述一个对象或系统在其生命周期内响应事件所经历的状态序列以及它对这些事件的响应。
 - 状态 (State)：对象在特定时间表现出的情况。
 - 状态转换 (Transition)：从一个状态到另一个状态的转移。
 - 事件 (Event)：导致状态转换的触发。
 - 活动 (Activity)：状态转换发生时或在某个状态内执行的过程。
- 建立步骤：
 - a. 确定上下文环境：明确状态图的主体（通常是某个重要对象或用例的执行过程）。
 - b. 识别状态：找出主体在其生命周期中所有可能的重要状态。
 - c. 建立状态转换：确定哪些事件会导致从一个状态迁移到另一个状态。
 - d. 补充详细信息：添加进入/退出活动、监护条件等。
- 注意：并非所有状态图都有明确的开始（通常一个）和结束态（可多个）。
- 论述题考点：
 - 状态图的应用场景和价值：论述何时需要为系统或对象建立状态图（例如，当对象行为复杂，受事件驱动明显时），以及状态图如何帮助理解 and 设计这类对象的行为。
 - 如何识别状态和转换：结合示例（如UC1销售处理），阐述从业务流程描述中提取关键状态点和触发转换的事件的思路和方法。

第七章 需求文档化与验证

5. 为什么需要需求规格说明 (SRS)?

- 沟通媒介：软件开发涉及多方协作，SRS是关键沟通和交流基础。
- 解决方案描述：从产品角度，以系统级需求列表方式描述软件解决方案。
- 与用例的区别：
 - 用例侧重交互流程。
 - SRS侧重独立需求和系统处理细节。
- 论述题考点：
 - SRS在软件开发生命周期中的核心作用：论述SRS如何作为后续设计、实现、测试和验收的基准，以及它对项目管理、质量保证的重要性。
 - SRS与用例文档的关系：阐述它们如何互补，用例提供上下文和用户视角，SRS提供精确、可验证的系统行为和特性描述。

6. 判定并修正需求规格说明示例的错误

- 需求文档化要点：
 - 技术文档写作：简洁、精确、易读、易修改。
 - 需求书写：用户术语、可验证、可行性。
 - SRS文档书写：标准模板、内容得当、完备性、一致性、优先级。
- 常见错误类型及修正思路 (结合示例)：
 - 模糊不清/不精确 (如“相关信息应被添加到日志文件” -> 应明确哪些信息，日志格式等)。
 - 不可验证 (如“很容易添加新功能” -> 如何衡量“容易”？应改为可测量的指标，如“添加新的支付方式不超过X人天”)。
 - 实现细节过早描述 (如“使用两位小数点表示加仑小数部分” -> 如果是业务规则可以保留，如果是纯实现细节则不妥)。
 - 过于理想化/不切实际 (如“一周7天，一天24小时都可用” -> 可能成本过高，应明确可用性级别，如99.9%)。
- 论述题考点：
 - 高质量需求的特性及如何达成：论述一个好的需求应具备的特性（如完整、一致、无歧义、可验证、可行、必要、可追踪等），并结合实际案例说明如何通过评审、度量等手段来保证这些特性。
 - 分析给定需求描述的缺陷并提出改进方案：这是非常典型的论述题，需要针对具体例子，指出其违反了哪些良好需求的原则，并给出修改后的版本。

7. 给定的需求示例，设计功能测试用例

- 核心思想：功能测试用例旨在验证软件是否满足SRS中定义的功能需求。
- 设计方法：主要基于黑盒测试技术。
 - 等价类划分：将输入数据划分为若干等价类，从每个类中选取代表性数据作为测试用例。
 - 边界值分析：选取等价类边界上、刚好在边界内和边界外的值作为测试用例。
 - 决策表：适用于有多个输入条件组合，每个组合对应不同动作的场景。
 - 因果图：更形式化的方法，分析输入（原因）和输出（结果）之间的关系。
- 论述题考点：
 - 如何基于需求规格说明设计有效的功能测试用例：阐述选择合适的黑盒测试技术（如等价类、边界值）的理由，并说明设计测试用例的步骤和关键考虑因素（如覆盖率、预期结果的明确性）。
 - 结合一个具体的功能需求描述，设计一组功能测试用例：例如，针对“会员积分兑换商品”功能，设计测试用例，包括有效输入、无效输入、边界条件等。

第8-11章: 软件设计与人机交互

第8章: 软件设计概览

1. 软件设计的定义

- 过程：为满足需求而定义系统/部件的体系结构、部件、接口等特征。
- 结果：设计过程的产物（如设计文档、模型）。

2. 软件设计的核心思想

- 复杂度控制 (Complexity Control)，通过：
 - 分解 (Decomposition)：将大问题分解为小问题。
 - 抽象 (Abstraction)：关注本质，忽略非本质细节。
 - 层次性 (Hierarchy)：组织模块形成层次结构。
- 论述题考点：论述分解、抽象、层次化这三个核心思想如何在软件设计（从体系结构到详细设计）中帮助我们控制复杂性。

3. 软件设计的三个层次

- 高层设计 (High-Level Design / Architectural Design)：定义系统整体结构，包括主要部件及其交互（连接件），选择体系结构风格。
- 中层设计 (Mid-Level Design / Module Design)：将部件划分为模块，定义模块接口和模块间关系，隐藏细节。
- 低层设计 (Low-Level Design / Detailed Design)：模块内部实现，数据结构和算法设计。
- 论述题考点：阐述软件设计三个层次的主要任务和关注点，以及它们之间的关系。

第9-10章: 软件体系结构设计 with 构建

4. 体系结构的概念

- 部件 (Component)：承载系统主要功能（处理与数据）。
- 连接件 (Connector)：定义部件间的交互方式（是连接的抽象）。
- 配置 (Configuration)：部件和连接件的关联方式，形成系统总体结构。

5. 体系结构的风格及其优缺点

- (1) 主程序/子程序风格 (Main Program/Subroutine)
 - 结构：层次化分解，单线程控制，子程序调用。
 - 优点：流程清晰，易理解，强控制性。
 - 缺点：调用是强耦合，难修改复用；可能通过共享数据交流导致公共耦合。
 - 适用：小型、控制流程简单的系统。

- (2) 面向对象式风格 (Object-Oriented Style)
 - 结构：对象作为部件，方法调用为连接件，数据封装。
 - 优点：内部实现可修改，易开发、理解、复用。
 - 缺点：接口耦合，标识耦合，副作用（多对象修改同一数据）。
 - 适用：核心是识别和保护相关数据（信息隐藏）的应用。
- (3) 分层风格 (Layered Style)
 - 结构：部件组织成层次，上层调用下层服务，禁止跨层和逆向调用。
 - 优点：设计清晰，支持并行开发，较好复用性和内部可修改性。
 - 缺点：交互协议难修改，可能有性能损失，层数和粒度难确定。
 - 适用：服务可分层组织的应用（如网络协议OSI七层模型）。
- (4) MVC风格 (Model-View-Controller)
 - 结构：
 - Model：领域知识和数据，通知View变化。不依赖View/Controller。
 - View：显示信息给用户，将用户手势（输入）发给Controller。
 - Controller：接收用户输入，改变Model状态，选择View响应。
 - 优点：易开发性，View和Controller可修改性好，适宜网络系统。
 - 缺点：复杂性较高，Model修改可能困难（若View/Controller对其有过多假设）。
 - 适用：用户界面易变或需多种表示，UI调整不应影响核心功能（如Web应用）。
- 论述题考点：
 - 选择2-3种常见的软件体系结构风格（如分层、MVC），详细阐述其结构特点、优缺点及典型应用场景。
 - 针对一个具体应用场景（如网上书店、学生管理系统），分析其需求特点，并为其选择一个合适的体系结构风格，论证你的选择理由。

6. 体系结构设计的过程

7. 分析关键需求和项目约束。

8. 选择体系结构风格，确定顶层架构。

9. 进行软件体系结构逻辑设计（抽象部件和接口）。

10. 依赖逻辑设计进行软件体系结构物理/具体设计（技术选型，映射到实现）。

11. 完善体系结构设计（细化接口，考虑非功能需求）。

12. 添加构建接口。

13. 迭代3-7。

14. 包的原则 (Package Design Principles)

- 包内聚性原则 (Release/Reuse Equivalence, Common Closure, Common Reuse):
 - REP (复用发布等价原则): 复用的粒度就是发布的粒度。一个包中的类应形成可复用和发布的模块。
 - CCP (共同封闭原则): 包中的所有类对于同一类性质的变化应该是共同封闭的。一个变化若影响一个包, 则应影响包中所有类, 但不影响其他包。
 - CRP (共同复用原则): 一个包中的所有类应该是共同复用的。如果你复用了包中的一个类, 那么你就复用了包中的所有类。
- 包间耦合性原则 (Acyclic Dependencies, Stable Dependencies, Stable Abstractions):
 - ADP (无环依赖原则): 包之间的依赖关系不应形成环。
 - SDP (稳定依赖原则): 朝着稳定的方向进行依赖。一个包不应依赖于比它自身更不稳定的包。
 - SAP (稳定抽象原则): 包的抽象程度应该与其稳定性程度一致。稳定包应是抽象包, 不稳定包应是具体包。
- 论述题考点:
 - 阐述包设计的重要性, 并选择几个关键的包设计原则 (如CCP, CRP, ADP, SDP), 解释其含义及如何指导实际的包划分。
 - 分析CCP与CRP之间的权衡关系, 以及在项目不同阶段如何侧重。(CCP使维护者方便, CRP使复用者方便; CCP使包大, CRP使包小)。

15. 体系结构构件之间接口的定义

- 依据分配的需求确定模块对外接口。
- 依据刺激与响应确定接口, 明确接口内容 (数据、返回值)。
- VIEW的required接口 -> Logic的provided接口。
- LOGIC的required接口 -> 分解为同层模块和DATA的provided接口。
- 解除双向依赖: 抽象为独立接口和包。

16. 体系结构开发集成测试用例: Stub和Driver

- Stub (桩): 模拟被调用模块, 供调用模块测试使用。简单实现接口, 返回预设数据。
- Driver (驱动): 模拟调用模块, 用于测试被调用模块。准备测试数据, 调用被测模块, 验证结果。
- 持续集成: 逐步用真实模块替换Stub/Driver。
- 论述题考点: 解释桩(Stub)和驱动(Driver)在自顶向下和自底向上集成测试策略中的作用, 并说明它们如何支持早期集成和测试。

第11章: 人机交互设计 (HCI)

17. 可用性 (Usability)

- 多维度质量属性，关注用户使用系统的过程及系统对用户产生的作用。
- 常用度量维度：
 - 易学性 (Learnability)：用户学习使用系统的难易程度。
 - 易记性 (Memorability)：用户在一段时间未使用后，再次使用时恢复熟练程度的难易程度。
 - 效率 (Efficiency)：用户熟练后完成任务的速度。
 - 低出错率 (Low Error Rate)：用户使用时犯错的频率和严重性，以及从错误中恢复的容易程度。
 - 主观满意度 (Subjective Satisfaction)：用户对使用系统的主观感受。
- 论述题考点：阐述可用性的五个关键维度，并说明为何在人机交互设计中需要综合考虑这些维度以提升用户体验。

18. 界面设计的注意事项及解释 (至少5个，黄金规则等)

19. 一致性 (Strive for consistency)：相似情境下操作和术语一致，界面元素风格统一。
20. 为常用用户提供快捷方式 (Enable frequent users to use shortcuts)：如键盘快捷键、宏、命令行。
21. 提供信息丰富的反馈 (Offer informative feedback)：对用户操作及时给出明确响应。
22. 设计对话框以产生结束信息 (Design dialogs to yield closure)：操作序列应有明确的开始、过程和结束。
23. 提供简单的错误处理 (Offer simple error handling)：预防错误，出错时提供清晰信息和简便恢复途径。
24. 允许方便地撤销操作 (Permit easy reversal of actions)：鼓励用户探索，减轻操作顾虑。
25. 让用户掌握控制权 (Support internal locus of control)：用户应感觉是操作的主导者，而非被动响应。
26. 减轻短期记忆负担 (Reduce short-term memory load)：界面信息应简洁明了，避免用户记忆过多信息。
27. (不要暴露内部结构)：用户界面应反映用户任务模型，而非系统实现模型。
28. (所见即所得 WYSIWYG)：操作结果应直观可见。

- 论述题考点：选择3-5条界面设计黄金规则（或尼尔森十大可用性原则），结合具体示例阐述其含义以及如何应用于实际界面设计中以提高可用性。

29. 精神模型与用户差异性

- 精神模型 (Mental Model)：用户头脑中关于系统如何工作以及如何使用它来完成任务的模型。
 - 隐喻 (Metaphor)：利用用户熟悉的现实事物（如桌面、购物车）来帮助构建精神模型。
- 用户差异性：
 - 新手用户：关注易学性，需清晰导航，避免错误。

- 专家用户：关注效率，希望有快捷方式。
- 熟练用户：在易学性和效率间平衡。
- 设计应提供差异化交互机制：如图形界面（GUI）为主，辅以命令行（CLI）、快捷键等。
- 论述题考点：论述精神模型在人机交互设计中的重要性，以及如何通过隐喻设计和考虑用户差异性来创建更易用的界面。

30. 导航、反馈、协作式设计

- 导航 (Navigation):
 - 全局结构：按任务模型组织功能，区分重要性和主题（窗口、菜单、列表、快捷方式）。
 - 局部结构：界面布局细节，视觉线索引导用户（控件布局、按钮、颜色字体）。
- 反馈 (Feedback):
 - 让用户意识到行为结果，不中断意识流。
 - 响应时间：短（<0.1s瞬时）、中（1s内不打断）、长（>10s需进度提示）。
- 协作式设计原则 (指良好交互的设计要点):
 - 简洁设计：摘要图片优于文字，列举、隐藏、赋予线索（标签、图标）。
 - 一致性设计：确认删除键位置等应一致。
 - 低出错率设计：限制输入（列表、可选框），限定输入（类型、范围、格式），简化操作，事前提示，事后检查，随时撤销。
 - 易记性设计：减少短期记忆负担，逐层递进，直观快捷方式，有意义的缺省值。
- 论述题考点：阐述导航和反馈在用户界面设计中的关键作用。如何通过良好的导航设计帮助用户高效找到功能，以及如何通过及时的反馈增强用户的控制感和系统透明度？

第12-15章: 详细设计

第12章: 详细设计的出发点与职责分配 (GRASP)

1. 详细设计的出发点

- 输入：需求开发成果（SRS、分析模型）和软件体系结构成果（架构设计方案、原型）。
- 目标：实现所有功能性+非功能性需求。
- 核心活动：明确职责建立静态模型（设计类图），明确协作建立动态模型（详细顺序图）。

GRASP (General Responsibility Assignment Software Patterns)

- 信息专家 (Information Expert):
 - 原则：将职责分配给拥有完成该职责所需信息的类。
 - 优点：促进低耦合、高内聚、维护封装。

- 示例：计算总价，Sale 类知道所有 SalesLineItem，SalesLineItem 知道数量和单价，以及应用的促销策略。所以 Sale 委托 SalesLineItem 计算小计，SalesLineItem 获取单价和促销信息。
- 控制器 (Controller):
 - 原则：处理系统外部事件（来自UI或外部系统），将职责分配给代表整个系统（门面控制器）或用例场景（用例控制器）的类。
 - 核心思想：解耦UI与业务逻辑。
 - 避免：UI直接调用业务逻辑，业务逻辑直接操作UI。
- 创建者 (Creator):
 - 原则：当B包含/组合A、B记录A、B密切使用A、B拥有A的初始化数据时，由B创建A的实例。
- 纯虚构 (Pure Fabrication):
 - 原则：当现有领域类都不适合承担某个职责（会导致高耦合或低内聚）时，创建一个不代表领域概念的人工类来承担该职责。
 - 示例：数据库访问对象 (DAO)、日志记录器、UI模型。
 - 作用：保持代码可复用性，实现高内聚、低耦合。
- 论述题考点：
 - GRASP原则在面向对象设计中的指导作用：选择2-3个GRASP原则（如信息专家、控制器），详细阐述其核心思想、应用场景、优点，并结合示例说明如何应用它们来指导职责分配，从而提高设计的质量（如低耦合、高内聚）。
 - 控制器模式如何实现界面与逻辑的解耦：详细解释控制器作为中间人的角色，如何接收界面请求，协调业务逻辑对象完成任务，并选择合适的视图进行展示，从而避免界面与业务逻辑的直接依赖。

2. 职责分配 (Responsibility Assignment)

- 类的职责：属性职责（拥有什么信息）和方法职责（能做什么）。
- 类间关系表达职责的划分与组合（依赖 < 关联 < 聚合 < 组合 < 继承）。

3. 协作 (Collaboration)

- 动态模型：对象如何交互以完成更大职责。
- 方法：自小而大（聚合小职责）或自大而小（分解大职责）。

4. 控制风格 (Control Style)

- 集中式：少数控制器掌握大部分系统行为逻辑。
- 分散式：行为逻辑广泛分布在对象网络中。
- 委托式 (Delegated)：决策分布，一些控制器做主要决策，并将部分职责委托给其他对象（通常是信息专家）。（推荐）

- 论述题考点：
 - 比较不同的控制风格及其对系统设计的影响：论述集中式、分散式和委托式控制风格的特点、优缺点，以及它们分别适用于哪些类型的系统或场景。强调委托式风格在平衡职责与协作方面的优势。
- 5. 建立设计类图或详细顺序图
 - 步骤：抽象类的职责 -> 抽象类间关系 -> 添加辅助类（接口类、记录/数据类、启动类、控制器类、实现数据类型的类、容器类）。
- 6. 协作的测试 (MockObject)
 - 使用Mock对象模拟协作者，以便独立测试一个类的逻辑。

第13章: 详细设计中的模块化与信息隐藏

- 7. 耦合 (Coupling) 与 内聚 (Cohesion)
 - 耦合：模块间关系的复杂程度。越低越好。
 - a. 内容耦合 (最高，不可接受)：一模块直接修改另一模块内部数据或代码。
 - b. 公共耦合 (不可接受)：多模块共享全局数据。
 - c. (重复耦合) (教材未明确列出，但实践中指模块间有重复代码，也应避免)
 - d. 控制耦合 (可接受，但需谨慎)：一模块传递控制信息给另一模块（如标志参数）。
 - e. 印记耦合 (Stamp Coupling) (可接受)：模块间传递复杂数据结构，但接收方仅使用其中一部分。
 - f. 数据耦合 (最理想)：模块间仅通过参数传递必需的数据项。
 - 内聚：模块内部联系的紧密性。越高越好。
 - a. 功能内聚 (最高，理想)：模块内所有元素共同完成一个且仅一个功能。
 - b. 信息内聚 (高)：模块执行多个操作，每个操作有独立入口点，但所有操作都在同一数据结构上进行（如一个类封装数据及相关操作）。
 - c. 通信内聚 (中等)：模块内各部分操作使用相同输入数据或产生相同输出数据。
 - d. 过程内聚 (中等)：模块内各部分按特定顺序执行。
 - e. 时间内聚 (较低)：模块内各部分因需在同一时间段执行而组合在一起（如初始化模块）。
 - f. 逻辑内聚 (低)：模块内包含一组逻辑上相似但功能不同的操作，通过参数选择执行其一（如一个数学库函数模块）。
 - g. 偶然内聚 (最低，不可接受)：模块内各部分无任何有意义的联系，仅随意组合。
 - 论述题考点：
 - 论述低耦合高内聚原则对软件质量的重要性：解释耦合和内聚的定义，为何追求低耦合高内聚（可维护性、可理解性、可复用性、健壮性），并结合具体耦合/内聚类型举例说明。

- 分析给定代码/设计片段的耦合与内聚程度，并提出改进建议：这是经典的题目，需要识别出不良的耦合（如公共耦合、内容耦合）或内聚（如偶然内聚、逻辑内聚），并运用设计原则（如信息专家、封装）进行重构以改善。

8. 信息隐藏 (Information Hiding)

- 基本思想：每个模块隐藏一个重要的设计决策（职责的实现细节、易变部分）。模块对外提供契约（接口），隐藏内部实现。
- 模块的秘密 (隐藏内容)：分配的职责、内部实现机制、容易变更的地方。
- 论述题考点：
 - 信息隐藏原则的含义及其在提高软件可维护性和可扩展性方面的作用：解释什么是信息隐藏，为什么要隐藏，隐藏什么，以及如何通过接口设计实现信息隐藏，从而隔离变化，降低修改带来的风险。

项目实践中的耦合、内聚、信息隐藏处理

- 耦合处理：分层（层间仅程序调用和简单数据传递）、包设计（消除重复、接口最小化）、创建者模式（不增加新耦合）、控制器模式（解耦View与Logic）。
- 内聚处理：分层（层间职责分配高内聚，层内分包高内聚）、信息专家模式、控制器与委托式控制风格。
- 信息隐藏处理：分层与分包（消除职责重复、最小化接口）、模块接口定义、类信息隐藏（协作设计、接口定义）、变化点设计（如RMI）。

第14章: 详细设计中面向对象方法下的模块化 (设计原则)

- SOLID原则及其他重要原则：
 - a. SRP (单一职责原则)：一个类应该只有一个引起它变化的原因。
 - b. OCP (开闭原则)：对扩展开放，对修改封闭。(通过抽象和多态实现)
 - c. LSP (Liskov替换原则)：子类型必须能够替换掉它们的基类型。
 - d. ISP (接口分离原则)：不应强迫客户端依赖于它们不使用的方法。多个特定客户端接口优于一个通用接口。
 - e. DIP (依赖倒置原则)：高层模块不应依赖低层模块，两者都应依赖抽象；抽象不应依赖细节，细节应依赖抽象。
 - f. 迪米特法则 (LoD - Law of Demeter / 最少知识原则)：一个对象应该对其他对象有尽可能少的了解。只与朋友交谈（直接成员、参数、创建的对象）。
 - g. 组合优于继承 (Composition over Inheritance)：优先使用组合/聚合来复用代码，仅当确实存在“is-a”关系且符合LSP时才使用继承。
 - h. 不要有代码重复 (DRY - Don't Repeat Yourself)。
 - i. 显式原则：代码应清晰表达其意图，避免不必要的隐藏（有时与可修改性冲突，需权衡）。

j. 全局变量是有害的。

- 论述题考点：

- 选择2-3个面向对象设计原则（如OCP, LSP, DIP, SRP），阐述其核心思想、目标及如何在实践中应用：结合代码示例说明违反这些原则可能导致的问题，以及遵循原则带来的好处。
- 分析给定的代码/设计片段，指出其违反了哪些设计原则，并提出修正方案：例如，一个巨大的类可能违反SRP；大量的if-else判断类型可能违反OCP；子类重写方法改变了父类行为的契约可能违反LSP。

第15章: 详细设计中面向对象方法下的信息隐藏 (封装、OCP、DIP)

9. 信息隐藏的含义 (回顾与深化)

- 隐藏两种设计决策：需求分配的职责（主要秘密）和实现机制变更（次要秘密）。
- OO机制：
 - 封装：封装类的职责，隐藏实现细节和预计变更。
 - 抽象类/接口 + 继承/实现：暴露稳定抽象接口，封装具体实现细节。

10. 封装 (Encapsulation)

- 含义1：数据与行为集中。
- 含义2：接口与实现分离（对外接口公开，内部实现隐藏）。
- 封装内容：
 - 数据与行为：除非必要，不设public方法/属性，避免不必要的getX/setX。
 - 复杂结构：不暴露内部复杂数据结构（如用Iterator代替直接暴露List）。
 - 其他对象：通过委托，不直接返回内部对象的引用（除非客户端已知）。
 - 子类信息 (LSP相关)。
 - 潜在变更 (OCP, DIP相关)：识别变化点，封装之。
- 论述题考点：
 - 论述封装在面向对象设计中的多重含义及其对软件质量的贡献：不仅是数据隐藏，还包括行为、结构、协作者和变化点的封装，以及这如何增强模块独立性、安全性和可维护性。

11. 开闭原则 (OCP - Open/Closed Principle)

- 核心：对扩展开放，对修改封闭。
- 实现：通常通过抽象（接口、抽象类）和多态实现。当需求变更时，通过增加新的具体类来实现扩展，而不是修改现有类的代码。
- 违反标志：大量的 `switch-case` 或 `if-else if` 语句检查对象类型。
- 特例：对象创建时（如工厂模式中的选择逻辑）有时难免。

12. 依赖倒置原则 (DIP - Dependency Inversion Principle)

- 核心：
 - 高层模块不应依赖于低层模块，两者都应依赖于抽象。
 - 抽象不应依赖于细节，细节应依赖于抽象。
- 实现：通过引入接口或抽象类，使高层模块依赖于此抽象，低层模块实现此抽象。从而“倒置”了传统自上而下的依赖关系。
- 解决 `new` 的问题：常与工厂模式结合，由工厂负责创建具体类的实例，高层模块仅依赖抽象接口。
- 论述题考点 (OCP & DIP 常一起考)：
 - 阐述开闭原则和依赖倒置原则的核心思想，并说明它们之间如何协同工作以提高系统的灵活性和可维护性：OCP是目标，DIP是实现OCP的重要手段之一。通过依赖抽象，使得系统可以在不修改高层模块的情况下，通过替换或增加新的低层实现来扩展功能。
 - 如何通过引入抽象（如接口或抽象类）来重构一个违反OCP或DIP的设计：给定一个具体场景（如不同支付方式、不同日志记录方式），展示如何应用这两个原则进行改进。

设计模式的应用 (结合第16-19章内容理解)

- 策略模式 (Strategy Pattern)：
 - 意图：定义一系列算法，将每个算法封装起来，并使它们可以相互替换。策略模式让算法的变化独立于使用算法的客户。
 - 结构：Context（上下文，维护一个Strategy引用），Strategy（策略接口），ConcreteStrategy（具体策略实现）。
 - 应用场景：多种行为变体，用 `switch` 选择行为。
 - 原则体现：SRP (Context和Strategy职责分离)，OCP (新增策略无需修改Context)，DIP (Context依赖Strategy抽象)，使用组合。
- 抽象工厂模式 (Abstract Factory Pattern)：
 - 意图：提供一个接口，用于创建一系列相关或相互依赖的对象，而无需指定它们具体的类。
 - 结构：AbstractFactory（抽象工厂接口，声明创建一系列产品的方法），ConcreteFactory（具体工厂，实现创建具体产品的方法），AbstractProduct（抽象产品接口），ConcreteProduct（具体产品）。
 - 应用场景：需要创建的产品族（一组产品一起使用），系统需要独立于产品的创建、组合和表示，希望提供产品库的接口。
 - 原则体现：封装，OCP+LSP+DIP。
 - 与工厂方法比较：工厂方法针对一个产品等级结构，抽象工厂针对多个产品等级结构（产品族）。

- 单件模式 (Singleton Pattern):
 - 意图: 确保一个类只有一个实例, 并提供一个全局访问点。
 - 实现: 私有构造函数, 静态私有实例变量, 公有静态 `getInstance()` 方法。
 - 原则体现: 封装, 但可能滥用导致全局状态问题。
- 迭代器模式 (Iterator Pattern):
 - 意图: 提供一种方法顺序访问一个聚合对象中的各个元素, 而又不暴露该对象的内部表示。
 - 结构: Iterator (迭代器接口, 定义遍历方法如 `hasNext()`, `next()`), ConcreteIterator (具体迭代器), Aggregate (聚合接口, 定义创建迭代器的方法 `createIterator()`), ConcreteAggregate (具体聚合)。
 - 原则体现: 针对接口编程, SRP, ISP, OCP+LSP+DIP。
- 论述题考点:
 - 选择一个或多个设计模式 (如策略模式、抽象工厂模式), 阐述其意图、结构、适用场景及优缺点。并说明该模式如何体现了面向对象的设计原则。
 - 给定一个设计问题场景, 运用合适的设计模式给出解决方案的类图 and 关键代码片段, 并解释为何选择该模式。

第22-23章: 软件开发过程模型

a. 软件生命周期模型阶段

- 需求工程 — 软件设计 — 软件实现 — 软件测试 — 软件交付 — 软件维护。

b. 解释与比较不同的软件过程模型

- 通用论述题框架: 对于每个模型, 阐述其核心思想/特征、优点、缺点、适用场景。比较时, 可围绕需求确定性、风险、项目规模、对变更的适应能力等维度。
- (1) 构建-修复模型 (Build-and-Fix)
 - 特征: 无规范, 缺乏分析、设计、测试, 文档缺失。
 - 适用: 极小型、质量要求不高、无需维护的个人项目。
 - 缺点: 几乎无法用于正规开发。
- (2) 瀑布模型 (Waterfall Model)
 - 特征: 阶段划分清晰 (需求->设计->实现->测试->维护), 文档驱动, 每个阶段需评审验证。
 - 优点: 管理简单, 阶段明确, 强调早期规划和文档。
 - 缺点: 对文档期望过高, 线性假设不符实际, 客户参与不足, 里程碑粒度粗, 难以适应需求变更。
 - 适用: 需求非常稳定且明确, 技术成熟, 复杂度适中。

- (3) 增量迭代模型 (Incremental/Iterative Model)
 - 特征：先完成核心框架和前景范围，然后分多个迭代并行开发，每个迭代是一个小瀑布。渐进交付。
 - 优点：符合实践，并行缩短时间，渐进交付加强反馈、降低风险。
 - 缺点：需开放式体系结构，若不确定性多则难以在开始确定前景。
 - 适用：需求相对清晰，但允许分批实现，可快速交付核心功能的项目。
- (4) 演化模型 (Evolutionary Model) (常指演化原型或更广义的迭代增量)
 - 特征：应对需求频繁变更或不确定性。多次迭代，并行瀑布。更强调通过早期版本获取反馈来驱动后续开发。
 - 优点：同增量模型，更适应需求变化。
 - 缺点：项目范围早期难确定，后续迭代可能忽略分析设计退化为构建-修复。
 - 适用：需求不稳定、不明确的大型复杂系统。
- (5) 原型模型 (Prototyping Model)
 - 特征：大量使用抛弃式原型来澄清不确定性（尤其是需求不明确时）。先构建原型，用户试用反馈，完善需求，然后基于明确需求再正式开发。
 - 优点：加强用户交流，提高满意度，适用于不确定性大的新颖领域。
 - 缺点：成本高，有时间和费用风险，可能不舍得抛弃原型导致差代码进入产品。
 - 适用：用户需求模糊，或技术方案有较大不确定性的项目。
- (6) 螺旋模型 (Spiral Model)
 - 特征：风险驱动。结合瀑布与原型，每轮迭代包括：制定计划、风险分析、实施工程、客户评估。
 - 优点：强调风险管理，尽早发现和解决高风险问题，减少损失。
 - 缺点：模型复杂，管理难度大，依赖风险评估的准确性。
 - 适用：高风险、大规模、复杂的软件系统。
- 论述题考点：
 - 比较瀑布模型与迭代/演化/敏捷类模型在应对需求变更方面的差异和适用性。
 - 针对一个具体的项目场景（如需求高度不确定、风险极高、或需求非常稳定），分析并选择最合适的开发过程模型，并阐述理由。
 - 论述风险驱动（如螺旋模型）或需求驱动（如原型、演化模型）的思想对现代软件开发的影响。
- c. 软件工程知识体系的知识域 (SWEBOK)
 - 需求、设计、构造、测试、维护、配置管理、工程管理、工程过程、工程工具和方法、质量。

- (了解即可, 作为背景知识)

第20-23章: 软件维护与开发过程模型

第20-21章: 软件维护

a. 软件维护的重要性

- 应对新需求: 保持软件的服务价值。
- 适应环境变化: 延长软件生命周期。
- 修正缺陷: 保证软件正常运行。
- 论述题考点: 论述软件维护在软件生命周期中的地位和挑战: 说明维护成本往往远超开发成本, 维护类型(纠错性、适应性、完善性、预防性), 以及导致维护困难的因素(文档缺失、设计不良、人员变动等)。

b. 开发可维护软件的方法

- 考虑可变性: 分析需求易变性, 为变更设计(如使用设计模式、遵循设计原则)。
- 为降低维护困难而开发: 详细技术文档并及时更新, 保证代码可读性, 维护需求跟踪链, 维护回归测试基线。
- 论述题考点: 如何通过优秀的设计和开发实践来提高软件的可维护性?: 结合信息隐藏、低耦合高内聚、设计原则、设计模式、良好编码规范、文档、自动化测试等方面进行阐述。

c. 演化式生命周期模型

- 阶段: 初步开发 -> 演化 -> 服务 -> 逐步淘汰 -> 停止。
 - 初步开发: 完成第一版(可部分需求)。
 - 演化: 持续增值, 满足更多需求, 修正缺陷。特征: 较好可演化性, 能实现业务价值。若可演化性降低或业务价值不足, 则进入服务或停止阶段。
 - 服务: 不再大规模增值, 主要修正缺陷, 维持使用。
 - 逐步淘汰: 不再提供服务, 用户可能继续使用, 但需容忍问题。考虑作为遗留资源。
 - 停止: 不再维护, 用户不再使用。
- 论述题考点: 阐述演化式生命周期模型的特点, 并分析其与传统瀑布模型在应对需求变化方面的差异。

d. 用户文档与系统文档

- 用户文档: 参考指南、操作教程(用户手册、联机帮助)。组织模式: 指导模式(任务驱动)、参考模式(随机访问)。
- 系统文档: 面向维护, 性能调整、权限控制、故障解决、软硬件配置、安全、备份容灾等。

e. 逆向工程与再工程

- 逆向工程：分析目标系统，标识部件及其交互，创建更高层抽象的系统表现（抽取需求与设计）。
- 再工程：检查和改造目标系统，用新模式及其实现复原。目的：改善理解、提高可维护性/复用性/演化性。活动：重新文档化、重组结构、转语言、改数据结构。
- 论述题考点：论述逆向工程和再工程在处理遗留系统时的作用、过程和挑战。

第22-23章: 软件开发过程模型

f. 软件生命周期模型阶段

- 需求工程 — 软件设计 — 软件实现 — 软件测试 — 软件交付 — 软件维护。

g. 解释与比较不同的软件过程模型

- 通用论述题框架：对于每个模型，阐述其核心思想/特征、优点、缺点、适用场景。比较时，可围绕需求确定性、风险、项目规模、对变更的适应能力等维度。
- (1) 构建-修复模型 (Build-and-Fix)
 - 特征：无规范，缺乏分析、设计、测试，文档缺失。
 - 适用：极小型、质量要求不高、无需维护的个人项目。
 - 缺点：几乎无法用于正规开发。
- (2) 瀑布模型 (Waterfall Model)
 - 特征：阶段划分清晰（需求->设计->实现->测试->维护），文档驱动，每个阶段需评审验证。
 - 优点：管理简单，阶段明确，强调早期规划和文档。
 - 缺点：对文档期望过高，线性假设不符实际，客户参与不足，里程碑粒度粗，难以适应需求变更。
 - 适用：需求非常稳定且明确，技术成熟，复杂度适中。
- (3) 增量迭代模型 (Incremental/Iterative Model)
 - 特征：先完成核心框架和前景范围，然后分多个迭代并行开发，每个迭代是一个小瀑布。渐进交付。
 - 优点：符合实践，并行缩短时间，渐进交付加强反馈、降低风险。
 - 缺点：需开放式体系结构，若不确定性多则难以在开始确定前景。
 - 适用：需求相对清晰，但允许分批实现，可快速交付核心功能的项目。
- (4) 演化模型 (Evolutionary Model) (常指演化原型或更广义的迭代增量)
 - 特征：应对需求频繁变更或不确定性。多次迭代，并行瀑布。更强调通过早期版本获取反馈来驱动后续开发。
 - 优点：同增量模型，更适应需求变化。

- 缺点：项目范围早期难确定，后续迭代可能忽略分析设计退化为构建-修复。
- 适用：需求不稳定、不明确的大型复杂系统。

- (5) 原型模型 (Prototyping Model)

- 特征：大量使用抛弃式原型来澄清不确定性（尤其是需求不明确时）。先构建原型，用户试用反馈，完善需求，然后基于明确需求再正式开发。
- 优点：加强用户交流，提高满意度，适用于不确定性大的新颖领域。
- 缺点：成本高，有时间和费用风险，可能不舍得抛弃原型导致差代码进入产品。
- 适用：用户需求模糊，或技术方案有较大不确定性的项目。

- (6) 螺旋模型 (Spiral Model)

- 特征：风险驱动。结合瀑布与原型，每轮迭代包括：制定计划、风险分析、实施工程、客户评估。
- 优点：强调风险管理，尽早发现和解决高风险问题，减少损失。
- 缺点：模型复杂，管理难度大，依赖风险评估的准确性。
- 适用：高风险、大规模、复杂的软件系统。

- 论述题考点：

- 比较瀑布模型与迭代/演化/敏捷类模型在应对需求变更方面的差异和适用性。
- 针对一个具体的项目场景（如需求高度不确定、风险极高、或需求非常稳定），分析并选择最合适的开发过程模型，并阐述理由。
- 论述风险驱动（如螺旋模型）或需求驱动（如原型、演化模型）的思想对现代软件开发的影响。

h. 软件工程知识体系的知识域 (SWEBOK)

- 需求、设计、构造、测试、维护、配置管理、工程管理、工程过程、工程工具和方法、质量。
 - （了解即可，作为背景知识）